SMART DOG COLLAR

MQTT Interface Specification

Engineering Communication Interface

Document Attribute	Value
Project	Smart Dog Collar / Pet Tracker
Document Version	1.2
Protocol	MQTT
Payload Format	JSON
Status	Engineering Baseline - topic and payload catalogue expanded

Version 1.2 focus: Adds a master MQTT Topic Catalogue, payload field definitions, units/ranges, message behaviour, command correlation, retry/idempotency guidance, retained-message policy, keep-alive guidance, OTA topic proposal, and protocol-versioning rules.

Document Control

Version	Date	Description
1.0	-	Initial MQTT documentation.
1.1	2026-08-19	Explicit backend/device destination hierarchy and cleaned command/response model.
1.2	2026-08-19	Expanded engineering interface specification with master topic catalogue and implementation rules.

1. Scope and Purpose

This specification defines the MQTT interface between Smart Dog Collar devices and the backend platform. It is intended to be a shared implementation reference for embedded firmware, backend services, mobile/web applications, integration testing, and production operations.

- Defines topic hierarchy, publishers, subscribers, QoS, retain behaviour and message triggers.
- Defines baseline JSON payload fields, data types, units and expected ranges.
- Defines command/response correlation and duplicate handling.
- Defines recommended MQTT session, security and reliability behaviour.
- Identifies proposed extensions that require final product decisions before release.

Normative language: The words shall/must indicate required behaviour; should indicates a recommendation; may indicates an optional implementation choice.

2. Topic Naming and Direction

```
pettracker/{destination}/{device_id}/{main_topic}/{sub_topic}
pettracker/{destination}/{device_id}/settings/{action}/{setting}
```

Element	Allowed / Example	Rule
destination	backend | device	backend = device-to-cloud; device = cloud-to-device
device_id	dog001	Unique, lowercase identifier; no spaces
main_topic	telemetry, status, event, command, response, settings, diagnostic	Lowercase
sub_topic	gps, battery, reboot, etc.	Use snake_case for multi-word names

Examples:

```
pettracker/backend/dog001/telemetry/gps
pettracker/device/dog001/command/reboot
pettracker/device/dog001/settings/set/gps_interval
```

3. Common Message Envelope

All JSON messages should use UTF-8 and valid JSON. Where applicable, the following common fields are used.

Field	Type	Required	Description
timestamp	integer	Yes	Unix epoch time in seconds, UTC.
request_id	string	Commands/settings	Unique correlation identifier generated by backend.

status	string	Responses	success | failed | rejected | unsupported.
error_code	string	On error	Machine-readable error identifier.
message	string	Optional	Human-readable diagnostic description; not for control logic.
protocol_version	string	Recommended	Payload contract version, e.g. 1.2.

```
{
 "timestamp": 1717330000,
 "protocol_version": "1.2"
}
```

Data typing: Booleans must be JSON booleans (true/false), timestamps must be numeric integers, and numeric measurements should not be encoded as strings.

4. Master MQTT Topic Catalogue

The following catalogue is the primary implementation reference. Frequencies are recommended defaults and may be overridden by product settings.

Cat.	Topic	Pub.	Sub.	QoS	Ret.	Trigger	Purpose
Telemetry	pettracker/ backend/ {device_id}/ telemetry/gps	Device	Backend	0	No	Periodic / requested	GPS fix and movement data
Telemetry	pettracker/ backend/ {device_id}/ telemetry/battery	Device	Backend	0	No	Periodic / state change	Battery voltage, SoC, charging
Telemetry	pettracker/ backend/ {device_id}/ telemetry/imu	Device	Backend	0	No	Configured interval / motion	Accelerometer and gyroscope
Telemetry	pettracker/ backend/ {device_id}/ telemetry/ signal_strength	Device	Backend	0	No	Periodic / network change	Cellular signal metrics
Status	pettracker/ backend/ {device_id}/ status/system	Device	Backend	0	Optional	Boot + periodic	Firmware, uptime, memory, reset reason
Status	pettracker/ backend/ {device_id}/ status/network	Device	Backend	0	Optional	Connect/change + periodic	Network state
Status	pettracker/ backend/ {device_id}/ status/ connectivity	Device/LWT	Backend	1	Yes	Connect/ disconnect	Online/offline state
Event	pettracker/ backend/ {device_id}/ event/geofence	Device	Backend	1	No	On event	Geofence enter/exit
Event	pettracker/ backend/ {device_id}/event/ low_battery	Device	Backend	1	No	Threshold crossing	Low battery alert
Event	pettracker/ backend/ {device_id}/event/ tamper	Device	Backend	1	No	On detection	Tamper alert
Command	pettracker/ device/ {device_id}/ command/location	Backend	Device	1	No	On demand	Request current location
Command	pettracker/	Backend	Device	1	No	On demand	Buzzer control

	device/{device_id}/command/buzzer						
Command	pettracker/device/{device_id}/command/reboot	Backend	Device	1	No	On demand	Restart device
Command	pettracker/device/{device_id}/command/peripheral	Backend	Device	1	No	On demand	Peripheral power/control
Command	pettracker/device/{device_id}/command/sleep	Backend	Device	1	No	On demand	Sleep/wake mode control
Response	pettracker/backend/{device_id}/response/location	Device	Backend	1	No	Command result	Location response
Response	pettracker/backend/{device_id}/response/buzzer	Device	Backend	1	No	Command result	Buzzer acknowledgement
Response	pettracker/backend/{device_id}/response/reboot	Device	Backend	1	No	Before reboot	Reboot acknowledgement
Response	pettracker/backend/{device_id}/response/peripheral	Device	Backend	1	No	Command result	Peripheral acknowledgement
Response	pettracker/backend/{device_id}/response/sleep	Device	Backend	1	No	Command result	Sleep acknowledgement
Settings	pettracker/device/{device_id}/settings/set/{setting}	Backend	Device	1	No	On configuration	Set one setting
Settings	pettracker/device/{device_id}/settings/get/{setting}	Backend	Device	1	No	On demand	Read one setting
Settings	pettracker/backend/{device_id}/settings/response/{setting}	Device	Backend	1	Optional	Set/get result	Setting value/result
Diagnostic	pettracker/backend/{device_id}/diagnostic/log	Device	Backend	0	No	As generated / throttled	Operational log
Diagnostic	pettracker/backend/{device_id}/diagnostic/crash	Device	Backend	1	No	After crash/reboot	Crash/reset information
Diagnostic	pettracker/backend/{device_id}/diagnostic/debug_log	Device	Backend	0	No	Debug builds / enabled	Verbose debug data
Diagnostic	pettracker/backend/{device_id}/diagnostic/sensor_health	Device	Backend	0	Optional	Periodic / fault change	Sensor health

5. Telemetry Payload Specification

5.1 GPS

pettracker/backend/{device_id}/telemetry/gps

Field	Type	Req.	Unit / Range	Description
timestamp	integer	Y	s since epoch	UTC sample time
latitude	number	Y	-90..90 deg	WGS84 latitude
longitude	number	Y	-180..180 deg	WGS84 longitude
speed	number	N	m/s, >=0	Ground speed
accuracy	number	N	m, >=0	Horizontal accuracy estimate
satellites	integer	N	>=0	Satellites used/visible as defined by firmware

```
{
 "timestamp": 1717330000,
 "latitude": 6.9271,
 "longitude": 79.8612,
 "speed": 1.2,
 "accuracy": 4.5,
 "satellites": 10
}
```

5.2 Battery

pettracker/backend/{device_id}/telemetry/battery

Field	Type	Req.	Unit / Range	Description
timestamp	integer	Y	s since epoch	UTC sample time
voltage	number	Y	V, >=0	Measured battery voltage
percentage	integer	Y	0..100 %	Estimated state of charge
charging	boolean	Y	true/false	Charging state
temperature	number	N	degC	Battery temperature if available

```
{
 "timestamp": 1717330000,
 "voltage": 3.95,
 "percentage": 82,
 "charging": false
}
```

5.3 IMU

pettracker/backend/{device_id}/telemetry/imu

Field	Type	Req.	Unit	Description
timestamp	integer	Y	s since epoch	UTC sample time
accelerometer.x/y/z	number	Y	m/s^2	Acceleration axes
gyroscope.x/y/z	number	Y	deg/s	Angular-rate axes

```
{
 "timestamp": 1717330000,
 "accelerometer": {"x": 0.12, "y": -0.03, "z": 9.78},
 "gyroscope": {"x": 0.4, "y": -0.2, "z": 0.1}
}
```

5.4 Cellular Signal

pettracker/backend/{device_id}/telemetry/signal_strength

Field	Type	Req.	Unit	Description
timestamp	integer	Y	s since epoch	UTC sample time
rssi	number	N	dBm	Received signal

				strength
rsrp	number	N	dBm	LTE reference signal received power
rsrq	number	N	dB	LTE reference signal received quality
sinr	number	N	dB	Signal-to-interference-plus-noise ratio

```
{
 "timestamp": 1717330000,
 "rssi": -79,
 "rsrp": -101,
 "rsrq": -11,
 "sinr": 14
}
```

6. Status and Event Payloads

6.1 System Status

```
pettracker/backend/{device_id}/status/system
```

```
{
 "timestamp": 1717330000,
 "firmware_version": "1.1.0",
 "uptime": 86400,
 "free_memory": 152320,
 "reset_reason": "power_on"
}
```

6.2 Network Status

```
pettracker/backend/{device_id}/status/network
```

```
{
 "timestamp": 1717330000,
 "connected": true,
 "network_type": "lte",
 "operator": "example_operator",
 "ip_address": "10.20.30.40"
}
```

6.3 Geofence Event

```
pettracker/backend/{device_id}/event/geofence
```

Field	Type	Req.	Allowed	Description
timestamp	integer	Y	epoch seconds	Event time
geofence_id	string	Y	Configured ID	Identifies geofence
event	string	Y	enter | exit	Transition type
latitude	number	N	-90..90	Position at event
longitude	number	N	-180..180	Position at event

```
{
 "timestamp": 1717330000,
 "geofence_id": "home",
 "event": "exit"
}
```

6.4 Low Battery Event

```
{
 "timestamp": 1717330000,
 "percentage": 15,
 "voltage": 3.55
}
```

6.5 Tamper Event

```
{
 "timestamp": 1717330000,
 "detected": true
}
```

7. Command and Response Contract

Every command that requires acknowledgement shall include request_id. The device shall echo that request_id in the corresponding response. QoS 1 can produce duplicate deliveries, therefore commands must be processed idempotently where possible.

Command	Request Payload	Response Topic	Expected Result
location	timestamp, request_id	response/location	status + location fields
buzzer	timestamp, request_id, enabled, duration	response/buzzer	status
reboot	timestamp, request_id	response/reboot	acknowledge before restart
peripheral	timestamp, request_id, peripheral, enabled	response/peripheral	status
sleep	timestamp, request_id, mode	response/sleep	status

7.1 Standard Command

```
{
 "timestamp": 1717330000,
 "request_id": "req_1003"
}
```

7.2 Standard Success Response

```
{
 "timestamp": 1717330001,
 "request_id": "req_1003",
 "status": "success"
}
```

7.3 Standard Failure Response

```
{
 "timestamp": 1717330001,
 "request_id": "req_1003",
 "status": "failed",
 "error_code": "reboot_failed",
 "message": "Unable to restart device"
}
```

7.4 Duplicate Command Handling

- The backend shall generate a unique request_id for each logical command.
- The device should cache recently processed request_id values for a defined period.

- If the same request_id is received again, the device should not repeat a non-idempotent physical action; it should return the previous result where practical.
- The backend should treat repeated responses with the same request_id as duplicates.

7.5 Timeout and Retry Baseline

Parameter	Recommended Baseline	Notes
Command acknowledgement timeout	10 s	Adjust for cellular latency and sleep state.
Backend retry count	2 retries	Use same request_id for retry of same logical command.
Retry interval	5-10 s	Consider exponential backoff.
Recent request_id cache	>= 5 min	Longer if commands can be delayed/offline queued.

Product decision: Final timeout/retry values should be validated against modem behaviour, device sleep strategy, broker session settings and expected cellular coverage.

8. Settings Contract

```
pettracker/device/{device_id}/settings/set/{setting}
pettracker/device/{device_id}/settings/get/{setting}
pettracker/backend/{device_id}/settings/response/{setting}
```

8.1 GPS Interval

Property	Definition
Setting name	gps_interval
Type	integer
Unit	seconds
Recommended allowed range	10..86400
Persistence	Non-volatile
Applies	After successful validation/write

```
{
 "timestamp": 1717330000,
 "request_id": "req_2001",
 "value": 45
}
```

8.2 Settings Response

```
{
 "timestamp": 1717330001,
 "request_id": "req_2001",
 "status": "success",
 "value": 45
}
```

Invalid settings should return status=failed or rejected with an error_code such as invalid_range, invalid_type, unsupported_setting, or storage_error.

9. Diagnostics and Error Model

Topic	QoS	Use
diagnostic/log	0	Normal operational logs; rate limit to protect bandwidth/power.
diagnostic/debug_log	0	Verbose logs; enable only when required.

diagnostic/crash	1	Crash/reset record that should be reliably delivered.
diagnostic/sensor_health	0	Periodic sensor availability/health state.

9.1 Diagnostic Log Payload

```
{
 "timestamp": 1717330000,
 "level": "error",
 "module": "storage",
 "error_type": "eeprom",
 "error_code": "400004"
}
```

9.2 Recommended Error-Code Pattern

Use stable machine-readable strings for application logic. Numeric vendor or subsystem codes may be included as additional fields.

```
{
 "status": "failed",
 "error_code": "invalid_range",
 "message": "gps_interval must be between 10 and 86400 seconds"
}
```

10. MQTT Session Behaviour

Item	Recommended Baseline
Protocol	MQTT 3.1.1 or MQTT 5.0; select one for production baseline
Transport	TLS encrypted TCP
Keep Alive	60 s starting baseline; optimise for modem/power behaviour
Clean session / clean start	Product decision based on offline command strategy
Telemetry retain	No
Event retain	No
Command retain	No
Response retain	No
Connectivity status retain	Yes
QoS 2	Not required in baseline

Commands must not be retained. A retained command could execute unexpectedly after a device reconnects.

11. Last Will and Connectivity

```
pettracker/backend/{device_id}/status/connectivity
```

After connecting, the device should publish a retained online state. The broker Last Will should publish a retained offline state if the connection terminates unexpectedly.

```
{
 "timestamp": 1717330000,
 "status": "online"
}

{
 "status": "offline"
}
```

12. Subscription and ACL Model

Client	Publish	Subscribe
Device	pettracker/backend/{its_device_id}/#	pettracker/device/{its_device_id}/#
Backend	pettracker/device/+/#	pettracker/backend/#

A device must not be able to publish or subscribe within another device's namespace. Broker ACLs should enforce this independently of application code.

Useful subscriptions:

```
pettracker/backend/#
pettracker/device/{device_id}/command/+
pettracker/device/{device_id}/settings/+/+
```

13. Security Requirements

- Use TLS and validate the broker/server certificate.
- Use unique device credentials or certificates; do not share one fleet-wide credential.
- Store credentials in protected device storage where hardware allows.
- Apply least-privilege broker ACLs.
- Do not place secrets, passwords, access tokens or private keys in MQTT payloads or topic names.
- Define credential rotation and device revocation procedures.
- Rate-limit diagnostic and command interfaces to reduce abuse and accidental battery drain.
- Backend should validate device_id, payload type, field ranges and message size before processing.

14. OTA / Firmware Update - Proposed Extension

Proposed: OTA topics are included as a design proposal for the next implementation phase and should not be considered enabled until firmware/backend behaviour and security are approved.

```
pettracker/device/{device_id}/command/firmware_update
pettracker/backend/{device_id}/response/firmware_update
pettracker/backend/{device_id}/status/firmware_update
```

Recommended OTA command payload:

```
{
 "timestamp": 1717330000,
 "request_id": "req_3001",
 "version": "1.2.0",
 "url": "https://example.invalid/firmware.bin",
 "sha256": "<64-hex-character-sha256>"
}
```

- Firmware shall be authenticated before installation; a hash alone is not an authenticity mechanism.
- Use signed firmware and verify the signature on the device.
- Use HTTPS/TLS for firmware transport.
- Report download, verification, installation and reboot status.
- Support rollback or recovery where the hardware/platform permits.

15. Protocol Versioning and Compatibility

- The MQTT topic root remains pettracker unless a breaking topic redesign requires a new root/version strategy.

- Payloads should include protocol_version when backend and firmware may evolve independently.
- Adding optional JSON fields is backward-compatible if receivers ignore unknown fields.
- Removing/renaming required fields or changing field meaning/unit is a breaking change.
- A breaking change should increment the major protocol version and be coordinated across firmware and backend.

Change	Compatibility
Add optional field	Backward-compatible
Add new topic	Backward-compatible
Add enum value	Potentially compatible; consumers must handle unknown values
Rename/remove required field	Breaking
Change unit or data type	Breaking
Change topic hierarchy	Breaking

16. Topic Hierarchy Summary

```
pettracker/
|-- backend/{device_id}/
|   |-- telemetry/{gps|battery|imu|signal_strength}
|   |-- status/{system|network|connectivity}
|   |-- event/{geofence|low_battery|tamper}
|   |-- response/{location|buzzer|reboot|peripheral|sleep}
|   |-- settings/response/{setting}
|   `-- diagnostic/{log|crash|debug_log|sensor_health}
`-- device/{device_id}/
    |-- command/{location|buzzer|reboot|peripheral|sleep}
    `-- settings/{set|get}/{setting}
```

17. Implementation Checklist

Area	Firmware	Backend / Cloud
Topic naming	Use exact lowercase topic names	Reject or log unexpected namespaces
JSON validation	Send correct types/ranges	Validate schema/types/ranges
QoS 1 duplicates	Deduplicate request_id	Deduplicate responses/events as needed
Commands	Never rely on retained commands	Never publish commands retained
Security	Validate broker certificate	Per-device ACL and credential management
Connectivity	Publish online + configure LWT	Track retained connectivity state
Settings	Validate and persist accepted values	Validate requested range before publish
Diagnostics	Rate-limit verbose logs	Apply ingestion limits/retention policy

18. Open Engineering Decisions

Decision	Why It Matters
MQTT 3.1.1 vs MQTT 5.0	Affects session semantics, reason codes and future features.
Final keep-alive value	Battery usage vs connection detection latency.
Offline command/session strategy	Determines clean session and queued QoS behaviour.
Exact telemetry intervals	Battery/data usage and tracking responsiveness.
GPS speed unit	Must be fixed across firmware/backend/UI.
IMU gyroscope unit	Must match sensor conversion and analytics.
Settings catalogue	Needs full list, type, range, persistence and defaults.
Geofence ownership	Device-side vs backend-side geofence calculation.
OTA architecture	Signing, download, rollback and deployment controls.
Payload/schema validation technology	JSON Schema or equivalent shared contract.

19. Quick Reference

```
Device -> Backend
pettracker/backend/dog001/telemetry/gps
pettracker/backend/dog001/telemetry/battery
pettracker/backend/dog001/status/system
pettracker/backend/dog001/status/connectivity
pettracker/backend/dog001/event/geofence
pettracker/backend/dog001/event/low_battery
pettracker/backend/dog001/response/reboot
pettracker/backend/dog001/settings/response/gps_interval
pettracker/backend/dog001/diagnostic/crash

Backend -> Device
pettracker/device/dog001/command/location
pettracker/device/dog001/command/buzzer
pettracker/device/dog001/command/reboot
pettracker/device/dog001/settings/set/gps_interval
pettracker/device/dog001/settings/get/gps_interval
```

20. Summary

Version 1.2 converts the MQTT guide into an engineering interface baseline. The master topic catalogue defines the expected publisher, subscriber, QoS, retain policy, trigger and purpose for each core topic. Payload definitions establish consistent data types and units, while command correlation, retry/idempotency, ACL, LWT, versioning and proposed OTA rules provide the implementation behaviour needed by firmware and backend teams.

Next recommended revision: Version 1.3 should freeze the complete settings catalogue, all enum values and error codes, exact telemetry intervals, MQTT protocol version/session policy, and formal machine-readable JSON Schemas for every payload.